



What is a CTE (Common Table Expression)?

A **CTE (Common Table Expression)** is a temporary named result set created using the '**WITH**' statement in SQL.

It allows you to write cleaner, more organized, and reusable queries, especially when working with complex SQL operations.

Think of a CTE as a temporary virtual table that exists only during the execution of a query.

Basic Syntax

```
WITH cte_name AS
(
    SELECT column_name
    FROM table_name
)
SELECT *
FROM cte_name;
```

Short Notes and Explanation of Each Script

SECTION 1: INTRODUCTION TO CTEs

```
USE techsphere_solutions;
```

What It Does

- Selects the techsphere_solutions database.
- Ensures all queries run inside the correct database.

Documentation

This is always the first step before querying tables.



SECTION 2: BASIC CTE EXAMPLE

```
WITH department_salary_stats AS
(
    SELECT
        d.department_name,
        SUM(jr.base_salary) AS total_salary,
        MIN(jr.base_salary) AS minimum_salary,
        MAX(jr.base_salary) AS maximum_salary,
        COUNT(jr.base_salary) AS total_roles,
        AVG(jr.base_salary) AS average_salary
```

What It Does

Creates a temporary dataset called department_salary_stats.

The Query Calculates

- Total salary per department
- Minimum salary
- Maximum salary
- Number of job roles
- Average salary

Documentation

This demonstrates how a CTE can simplify complex calculations and reporting.

```
SELECT *
FROM department_salary_stats;
```

What It Does

Displays all data stored in the temporary CTE.

Documentation

The CTE behaves like a temporary table inside the query.



SECTION 3: USING CTE VALUES FOR FURTHER ANALYSIS

```
WITH department_salary_stats AS
(
    SELECT
        d.department_name,
        SUM(jr.base_salary) AS total_salary,
        COUNT(jr.base_salary) AS total_roles
```

What It Does

Creates a summarized salary dataset grouped by department.

Documentation

This avoids repeating long calculations multiple times.

```
SELECT
    department_name,
    ROUND(total_salary / total_roles, 2) AS calculated_avg_salary
```

What It Does

Uses values from the CTE to manually calculate average salary.

Documentation

Demonstrates how CTE output can be reused in another query.

SECTION 4: MULTIPLE CTEs

```
WITH recent_employees AS
```

What It Does

Creates a temporary dataset for employees hired after 2022.



Documentation

Useful for HR and recruitment analytics.

```
high_salary_roles AS
```

What It Does

Creates another temporary dataset for high-paying job roles.

Documentation

Filters job roles with salaries above a specified amount.

```
SELECT
    re.employee_id,
    re.first_name,
    re.last_name,
    hsr.role_title,
    hsr.base_salary
```

What It Does

Joins the two CTEs together.

Result

Shows:

- Recently hired employees
- Their high-paying job roles
- Associated salaries

Documentation

Demonstrates how multiple CTEs can work together.



SECTION 5: RENAMING CTE COLUMNS

```
WITH salary_summary  
(  
    department_name,  
    total_salary,  
    minimum_salary,  
    maximum_salary,  
    total_roles  
)
```

What It Does

Renames CTE output columns directly.

Documentation

Makes queries cleaner and easier to understand.

```
SELECT  
    department_name,  
    ROUND(total_salary / total_roles, 2) AS avg_salary
```

What It Does

Calculates average salary using renamed CTE columns.

Documentation

Improves readability and removes the need for complex column names.

SECTION 6: PRACTICAL BUSINESS ANALYTICS

```
WITH department_costs AS
```

What It Does

Creates a temporary table containing total payroll cost per department.

Documentation

Useful for budgeting and financial analysis.



```
SELECT *  
FROM department_costs  
WHERE total_department_salary > 200000  
ORDER BY total_department_salary DESC;
```

What It Does

Filters departments with high salary expenses.

Result

Returns departments whose payroll exceeds 200,000.

Documentation

Useful for:

- HR budgeting
- Payroll monitoring
- Financial reporting

KEY CONCEPTS USED

Concept	Meaning
WITH	Starts a CTE
AS	Defines the temporary query block
CTE	Temporary named result set
Multiple CTEs	Multiple temporary datasets separated by commas
GROUP BY	Groups records
Aggregate Functions	SUM, AVG, MIN, MAX, COUNT

Advantages of CTEs

1. Improves Readability

Complex queries become easier to understand.

2. Makes SQL Modular

You can split large queries into smaller logical parts.



3. Reusable Logic

Avoid repeating the same subquery multiple times.

4. Easier Debugging

You can isolate sections of a query for troubleshooting.

5. Better for Analytics

Widely used in:

- Data analysis
- Reporting
- Dashboards
- KPI systems
- ETL pipelines

Simple Real-World Example

A company may use CTEs to:

- Calculate monthly sales summaries
- Identify top-performing employees
- Analyze payroll costs
- Generate dashboard metrics
- Prepare reporting datasets

Final Summary

This script demonstrates:

- Creating single and multiple CTEs
- Performing salary analytics
- Using temporary datasets
- Improving query readability
- Building reusable SQL logic
- Applying CTEs in real business analytics scenarios